
EMPLOYMENT MANAGEMENT SYSTEM

EMS Documentation Index

How to read this guide and the vocabulary it uses

INDEX AND GLOSSARY • 2 OCTOBER 2026

Contents

01	1. What the application is	3
02	2. Document reading order	3
03	3. The words you must know first	4
04	4. Repository layout	5
05	5. The three roles of every backend class	6
06	6. The PDF edition	6
07	7. How to verify anything you read here	7

Complete, code-level explanation of the Employment Management System (EMS).

This folder is written so that after reading it you can explain **any small detail** of the application: what happens on screen, what HTTP request is sent, which backend class runs, which SQL happens, and why the response looks the way it does.

1. What the application is

EMS is a two-part application:

Part	Technology	Repo
Backend (REST API)	Java 17, Spring Boot 4.0.2, Spring Data JPA, Spring Security, JWT, MySQL	<code>employment-management-system-backend</code>
Frontend (SPA)	React 19, Vite, React Router, TanStack Query, Zustand, Tailwind CSS	<code>employment-management-system-frontend</code>

The frontend is a *single-page application*. It never talks to MySQL directly. It only makes HTTP calls to the backend, and the backend is the only thing that talks to MySQL.

```
Browser (React + Zustand + TanStack Query)
  | fetch("/api/ ... ") + Authorization: Bearer <JWT>
  v
Spring Boot (Security filter chain → Controller → Service → Repository)
  | JDBC via Hibernate
  v
MySQL (employee_management_system)
```

2. Document reading order

Read them in this order. Each document assumes the previous one.

#	Document	What it covers
00	<code>00-README.md</code> (this file)	Index, glossary of the words used everywhere else
01	<code>01-foundations.md</code>	Layers, one HTTP request end-to-end, the database schema, <code>pom.xml</code> , <code>application.yaml</code> , startup + seeding
02	<code>02-authentication.md</code>	JWT, <code>SecurityConfig</code> , the JWT filter, login / register / me / logout flows
03	<code>03-employees.md</code>	Employee list, search, create, update, delete, department dropdown
04	<code>04-departments.md</code>	Department list, create, update, delete, view details

#	Document	What it covers
05	05-attendance.md	Attendance status rules, check-in, check-out, search, my-records, the seeder's history
06	06-frontend.md	How the React app is wired: router, authStore , uiStore , TanStack Query, every page
07	07-security-model.md	Roles, the complete endpoint permission matrix, where authorization is enforced
08	08-errors-and-glossary.md	HTTP status codes, GlobalExceptionHandler , remaining gaps, full glossary
09	09-round-trip-traces.md	End-to-end traces: all 18 flows, browser → HTTP → security → service → SQL → response → screen

3. The words you must know first

These terms are used constantly in the rest of the docs.

Flow / use case / endpoint

- **Endpoint** = one single HTTP address + method. Example: [GET /api/employees/paged](#). The backend code for it is one method in one controller.
- **Use case** = one thing a user wants to accomplish. Example: "see the employee list".
- **Flow** = the complete user journey for a use case, which may include typing, clicking, one or more HTTP requests, database queries, and what appears on screen.

The application has **21 endpoints** grouped into **18 user flows** across 5 features (Auth, Employees, Departments, Attendance, Dashboard).

What "end-to-end" means in this folder: a flow is *not* finished when the HTTP response arrives. Each flow is traced in **both directions**:

- **outbound** - the click, the React state, the [fetch\(\)](#) in [src/api/http.js](#), the JWT filter, [@PreAuthorize](#), the controller, the service, the repository, the SQL;
- **inbound** - the DTO/JSON, [useQuery](#) / [useMutation](#) receiving it, cache invalidation, [pushToast](#), [navigate](#), and the re-render that the user sees.

The full flow list is in [08-errors-and-glossary.md](#); every flow traced in both directions is in [09-round-trip-traces.md](#).

Roles

- [ROLE_ADMIN](#) - full control.
- [ROLE_MANAGER](#) - everything an admin can do in this app.
- [ROLE_EMPLOYEE](#) - only sees and manages their own attendance.

`ADMIN` and `MANAGER` behave identically here; they are separated only for future use. Both are called **manager-level** throughout the docs.

Authentication vs authorization

- **Authentication** = "who are you?" -> proven by a valid JWT.
- **Authorization** = "what may you do?" -> proven by your roles.

Both happen in the backend. Hiding a button in the frontend is *not* authorization; it is only convenience.

The word "employee" is overloaded

In this codebase `Employee` means **the HR record of a person** (name, email, salary, hire date, department). `User` means **the login account** (username, password hash, roles). One `User` links to at most one `Employee`, and one `Employee` links to at most one `User`. A manager's login account is *not* required to have an `Employee` record.

4. Repository layout

Backend (`employment-management-system-backend`):

```
src/main/java/com/example/ems/
  EmsApplication.java      entry point
  security/                JWT + user lookup
  security/config/SecurityConfig  the security filter chain
  auth/                    login / register / me
  user/, user/role/        accounts and roles
  employee/                employee records
  department/              departments
  attendance/              attendance records
  common/                  DataSeeder + error handling
```

TEXT

Frontend (`employment-management-system-frontend`):

```
src/
  main.jsx                 React root + QueryClientProvider
  App.jsx                  router definition
  api/http.js              every fetch call in the app
  store/authStore.js       token + current user
  store/uiStore.js         sidebar, theme, toasts
  hooks/usePermissions.js  role → canManage
  components/              Layout, RequireAuth, modals, Toasts
  pages/                   Login, Register, Dashboard, Employees, Departments, Attendance
```

TEXT

5. The three roles of every backend class

The single most useful thing to internalise:

```

Controller → reads the HTTP request, returns the HTTP response.
             Contains NO business logic.

Service     → the business rules.
             Contains NO HTTP concepts (no HttpServletRequest, no @RequestBody).

Repository  → reads and writes database rows.
             Contains NO business logic.

Mapper      → converts Entity ↔ DTO so the database shape
             never leaks into the JSON.
  
```

If you can answer "which of these four is this class?" for any file, you already understand most of the architecture.

6. The PDF edition

The same ten documents are published as styled A4 PDFs in `docs/pdf/`, one file per document, 126 pages in total. Each PDF has a cover page, a contents page with real page numbers, syntax-highlighted code, styled tables, running headers, and page numbers in the footer.

The Markdown in this folder is the source of truth. To rebuild the PDFs after editing it:

```

cd docs/tools
npm install      # once; needs a local Chrome or Edge
npm run check  # build, then verify contents page numbers, then layout QA
  
```

Script	What it does
<code>npm run build</code>	Renders every <code>docs/*.md</code> to <code>docs/pdf/*.pdf</code>
<code>npm run verify</code>	Confirms each contents page number is the page the heading is really on
<code>npm run qa</code>	Flags missing folios, near-empty pages, and headings stranded at a page foot
<code>npm run scan</code>	Fails if a secret value ever reappears in the rendered PDFs
<code>npm run check</code>	All of the above, in order

Values from `application.yaml` (the database password and the JWT signing secret) are redacted in these documents, so the PDFs can be shared safely.

Files that control the output:

File	Role
<code>docs/_pdf-theme.css</code>	The print stylesheet: cover, contents, headings, code, tables, callouts
<code>docs/_pdf-toc.js</code>	Runs in the page: heading ids, contents page, table headers, callouts
<code>docs/tools/build-pdf.mjs</code>	marked + highlight.js + headless Chrome

Both theme files live in `docs/` so the styling sits next to the documents it styles. Set `CHROME_PATH` if your browser is installed somewhere unusual.

7. How to verify anything you read here

- Build the backend: `.\mvnw.cmd clean package -DskipTests`
- Lint the frontend: `npx oxlint src`
- Build the frontend: `npm run build`
- Run the backend: `.\mvnw.cmd spring-boot:run` (port 8080)
- Run the frontend: `npm run dev` (Vite dev server, proxies `/api` to 8080)
- Rebuild the PDFs: `cd docs/tools && npm run check`